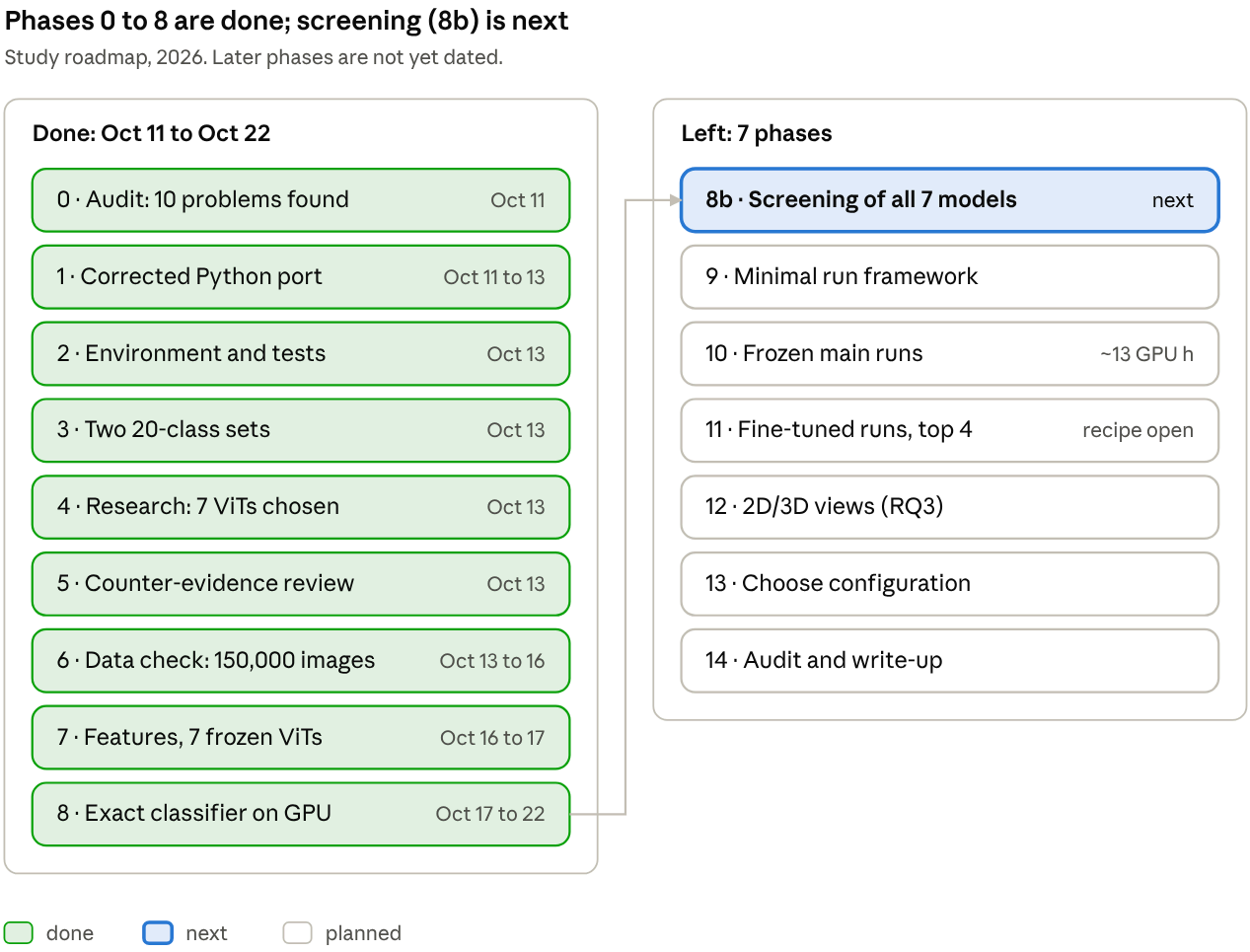
ViT Features + Pin-GTSVM on ImageNet-C — Project Report

Oct 22, 2026 · @Pranav Vaish

The study replaces the CNN feature extractors of a published hybrid classifier (Pin-GTSVM, arXiv:2501.13422) with 7 vision transformers and tests it on ImageNet-C, a benchmark of 15 image corruptions at 5 severities. Phases 0–8 of 14 are done: the data is checked, features from all 7 frozen models are extracted, and the paper's exact classifier now runs on a Kaggle GPU, matching the reference solver on 99.998 % of predictions. Next are the model screening and the main experiments, budgeted at about 13 GPU hours.

Roadmap



Study roadmap · 9 phases done, 7 left

Phases 0–8 took 12 days (Oct 11–22); the seven remaining phases are not yet dated, and the main frozen-model runs are budgeted at about 13 GPU hours.

Goal and research questions

The study asks how well modern vision transformers (ViTs), used as feature extractors, support a noise-robust classifier when images are corrupted. It is a new case study of the paper's hybrid idea (pretrained network + Pin-GTSVM), not an extension of the paper: the parking dataset, CNNs and haze are replaced by ImageNet-C, ViTs and 15 corruption types.

ID	Question
RQ1	How well do features from different ViTs (frozen and fine-tuned) support Pin-GTSVM, per corruption type and severity?
RQ2	Is Pin-GTSVM (pinball loss, $\tau > 0$) less sensitive to corruption noise than the same model without it ($\tau = 0$) and a standard SVM?
RQ3	Which 2D/3D views of the features and decision surfaces are faithful enough, by measured scores, to explain RQ1–RQ2?
RQ4	How does class similarity change the effect of corruptions: 20 similar classes (terriers) vs 20 unrelated classes?
RQ5	Which configuration gives the best robustness–efficiency trade-off for scale-up and edge devices?

Hypothesis for RQ2, from theory (Huang, Shi & Suykens, IEEE TPAMI 2014): the pinball loss ties the decision surface to a quantile of the class distances rather than the closest points, so accuracy should drop less from severity 1 to 5 when $\tau > 0$. It does not remove noise from the features; it changes how the surfaces respond to it.

Working principles set by the requester: thorough tests on data subsets rather than fewer checks; every change recorded; least system complexity and edge-deployable; all heavy computation on Kaggle (the local PC runs only seconds-long unit tests).

Starting point: the paper, its code and the corrected port

The source paper (Mehendale, Sahoo, Roul, arXiv:2501.13422, 2025) classifies hazy parking slots with frozen CNN features (ResNet-50, AlexNet, GoogLeNet) and a Pin-GTSVM with an RBF kernel. An audit of its MATLAB code (Phase 0) found 10 problems, so the method was first rebuilt in Python and corrected (Phase 1) before any new experiment.

#	Finding in the original code	Effect	Fix in the Python port
1	Hyperparameters chosen on the test set	Reported accuracies optimistically biased	Cross-validation on the training set; the paper's protocol kept only as a labelled comparison
2	$\tau = 1$ sets the upper bound $(1-\tau)c$ to 0	Every sample predicted +1	Box $-\tau c \leqslant \lambda \leqslant c$ from Tanveer et al. (2019)
3	Linear-kernel branch drops a test column	Crash	Fixed
4	Wrong quadratic program (no lower bound)	Not the published Pin-GTSVM	Correct dual, checked against an independently solved primal
5	Distances not divided by $\|w\|$	Biased class assignment	Normalised distances
6	False-positive list taken from the last grid point	Wrong images analysed	Uses the selected model
7	Unseeded CV, no standardisation, possible NaN precision	Not reproducible	Seeded stratified CV; NaN reported
8	Paper vs code mismatches (layers, table swap, GPU claims)	Inconsistent reporting	Documented
9	ResNet-50 features taken from class scores (fc1000)	Unusual feature choice	Pooled 2048-d layer offered

#	Finding in the original code	Effect	Fix in the Python port
10	MATLAB Caffe weights differ from torchvision weights	Numbers cannot match exactly	Documented

The port (`python_port/`, package `hazypark`) passes 67 tests; its dual solution matches an independently solved primal to 1e-5 for linear and RBF kernels at $\tau \in \{0, 0.5, 1\}$. It is the reference implementation of the paper's method for everything that follows.

Key decisions and why

Every choice below was proposed with its evidence and approved before work went ahead; the full log with dates is in `docs/scope.md`. Listed in the order they were made.

Date	Decision	Alternatives considered	Why
2026-10-11	Port the paper's code in corrected form only; τ as in Tanveer et al. (2019); CVXOPT as the exact solver	A faithful copy including the bugs	The original's test-set tuning and $\tau = 1$ collapse would make any comparison meaningless
2026-10-13	A new study on ImageNet-C, not an extension of the paper	More parking datasets	ImageNet-C has controlled corruption types and severities, which is what a noise study needs
2026-10-13	Use the ViTs both frozen and fine-tuned	Frozen only	Separates what the pretrained model already knows from what training on corruptions adds (RQ1)
2026-10-13	One-vs-one multiclass (190 binary classifiers per 20 classes)	One-vs-rest	Recommended for kernel SVMs (Hsu & Lin, IEEE TNN 2002); balanced binary problems

Date	Decision	Alternatives considered	Why
2026-10-13	Two 20-class sets: 20 terrier breeds (WordNet Wu-Palmer) and 20 unrelated classes (BREEDS hierarchy, farthest-first)	100 classes	100 classes = 4,950 classifier pairs, too costly; two contrasting sets answer RQ4 directly
2026-10-13	70/30 train/test split by source image (35/15 per class), 5 CV folds	Random image split	Each source image appears in all 75 corrupted versions; a random split would leak test images into training
2026-10-13	7 backbones: DINOv2 S/B, DINOv3 S/B, SigLIP 2 B, FastViT SA12/SA24	Larger ViTs (L, g), CNN controls	Best published frozen robustness on ImageNet-C (DINOv3), a common baseline (DINOv2), a different training signal (SigLIP 2), edge-sized models (FastViT)
2026-10-13	Add a held-out corruption family, linear-probe and kNN baselines, L2 vs standardisation, per-corruption reporting, Demšar statistics	Main grid only	Answers the strongest published criticisms of this kind of study (`docs/research_counterevidence.md`)
2026-10-13	All computation on Kaggle; the PC only for seconds-long unit tests	Local GPU (RTX 3050, 4 GB)	Data size (67.8 GB), GPU memory and the user's PC availability
2026-10-16	Close the data check without the byte comparison against the official Zenodo copy	Keep retrying the 6.6 GB download	Two download failures; completeness and structure were verified twice; reported as a limitation

Date	Decision	Alternatives considered	Why
2026-10-17	Pass mark for any faster solver: ≥ 99 % identical predictions and ≤ 0.5 points accuracy vs the exact method	No formal check	A speed-up must not change the study's results
2026-10-17	Screen all 7 models first, with a drop rule fixed in advance; fine-tune only the top 4	Full grid on all 7	Saves GPU time without hand-picking models after seeing results
2026-10-19	Drop the reduced-kernel (landmark) shortcut; use the paper's exact method on a GPU	Accept 98.5 % agreement	It failed the pass mark and was only ~4× faster
2026-10-20	Grid: σ = median distance × {¼, ½, 1, 2, 4}; c ∈ {2^{-5} … 2^{5}} (6 values); τ ∈ {0, 0.5, 0.8, 1}; 3 validation folds	180 settings, 5 folds	The full grid needed ~30 GPU hours per model and class set; the reduced one keeps the paper's range of c and τ
2026-10-21	Shared hyperparameters per model and class set, tuned on 20 seeded class pairs	All 190 pairs	~13 instead of ~100 GPU hours; one setting for all pairs is LIBSVM's standard for one-vs-one

Results so far: data and features

Data (Phase 6, Kaggle CPU) — passed twice, identically. The 40 classes give 150,000 images (40 classes × 50 source images × 15 corruptions × 5 severities), split 105,000 / 45,000 with no source image on both sides. Two runs on different Kaggle machines

produced byte-identical image lists, so the split is exactly reproducible (fingerprint `b3757fff…136a`). Limitation: the third-party Kaggle copy was checked for completeness and structure, not byte-for-byte against the official release.

Features (Phase 7, Kaggle T4, ~45 min) — all checks passed. All 7 models were used frozen (weights unchanged) at 224 × 224 pixels; each image became one summary vector (384–1,024 numbers). The features stay on Kaggle as the output of the notebook `phase7_features_v3`. A quick sanity classifier (logistic regression, mildest corruption level) checked that each extractor works:

Model	Params (M)	Compute (GMACs)	CPU ms / image	GPU ms / image	Terriers	Unrelated
FastViT-SA12	10.5	1.5	40	7.3	0.751	0.950
FastViT-SA24	20.5	2.9	76	13.2	0.787	0.952
DINOv2-S/14	21.6	5.5	77	6.6	0.741	0.969
DINOv3-S/16	21.6	4.3	65	10.2	0.722	0.966
DINOv2-B/14	85.7	21.9	240	7.7	0.789	0.974
DINOv3-B/16	85.6	17.2	205	10.3	0.788	0.979
SigLIP2-B/16	92.9	17.0	203	7.2	0.666	0.976

Accuracy is the share of test images classified correctly at severity 1; latency is one image at a time on Kaggle's T4 (fp16) and 2 CPU threads (fp32).

- Unrelated classes are near-perfect for every model, so the terriers are where models differ.
- FastViT-SA24 ties the large B models on terriers at one-seventh of the compute. Caveat to state in the paper: FastViT was trained with ImageNet labels, which include these breeds (training images only, so no test-image overlap).
- SigLIP 2 is last on terriers but near the top on unrelated classes: it learned from captions, which rarely name the breed.
- The summary (CLS) vector beat the average of patch vectors by 3–15 points on terriers, so it is the main feature vector.
- Differences under about 3 points are not meaningful here: the test set is 300 source images, each in 15 corrupted versions.

Phase 8: a classifier fast enough, without changing it

The paper's exact solver needs ~17 s per class pair on Kaggle's CPU, plus ~5.5 s per extra hyperparameter setting — months of computing for the planned grid. After seven checks against a fixed pass mark (≥ 99 % identical predictions, ≤ 0.5 points accuracy), the paper's **exact** method runs on a Kaggle T4 GPU, matching the CVXOPT reference on 99.998 % of about 6.6 million test predictions. Each check covered all 190 pairs of both class sets at three kernel widths plus extreme settings, on two models (DINOv3-S, FastViT-SA24).

Check	Date	Approach	Result	Lesson
v1	2026-10-17	Shortcut: compare with landmark images shared by all classes (CPU)	Failed: best 97.6 % identical	Only 1 in 10 landmarks belonged to the two classes compared
v2	—	Landmarks per class, reference on a P100 GPU	Not run	Kaggle no longer offers the P100
v3	2026-10-18	Landmarks per class, up to 500 each (CPU); exact CVXOPT answers saved	Failed: best 98.5 %, only ~4× faster	Shortcut dropped; the saved answers became the reference for later checks
v4	2026-10-20	Exact method on a T4 in single precision	Crashed	Single precision rounds the key matrix by ~0.3, more than its 0.05 safety margin
v5	2026-10-20	Exact method, numerically safe recipe (QR factor, double-precision finish)	**Passed: 99.998 % / 99.98 % identical, accuracy equal**	The method itself is now validated
v6	2026-10-21	Many problems solved together (batching)	Exact but slower; out of time	The GPU was already busy; my 3–10× speed estimate had not been measured

Check	Date	Approach	Result	Lesson
v7	2026-10-22	Fixed batching (finished problems leave the batch, stalled ones stop) + cost benchmark	**Passed (99.98–99.99 %), faster than v5**	Cost of the main runs measured

Measured cost of the approved tuning plan on a T4: 0.57–1.40 h per model and class set (mean 0.91 h), **about 12.8 GPU hours for all 7 models × 2 class sets**, within one week of Kaggle's ~30-hour quota. About 2 % of the optimisation problems stop just short of full accuracy without changing any result; all use the pinball loss ($\tau > 0$), mostly with weak regularisation ($c = 32$) — a finding to report.

Cost of the detours: about 30 Kaggle CPU hours (v1: 13.2 h, v3: 16.7 h) and about 5.5 T4 hours (v4–v7). Every failure was recorded, and each run was designed to save its finished results even when it failed.

Where we are and what is left

Phase 8 is finished; the next step is the screening of all 7 models (Phase 8b). No accuracy result of the study itself exists yet — everything so far builds and validates the pipeline.

Phase	Work	Compute	Status
8b	Screening: all 7 models, every corruption × severity, both class sets; linear probe, kNN and Pin-GTSVM at one default setting; drop rule fixed in advance	CPU + ~1–2 T4 h (estimate)	Next
9	Minimal experiment framework: one notebook per step, every run appended to a registry	—	With 8b/10

Phase	Work	Compute	Status
10	Main frozen-model runs: tune on 20 pairs, fit all 190 pairs at the best setting and at $\tau = 0$; evaluate per corruption, severity and family, held-out family, baselines, Demšar statistics	~0.9 T4 h per model and class set (measured)	Planned
11	Fine-tuned models, top 4 from screening; same protocol	Not yet estimated	Recipe to decide
12	2D/3D views of the features and decision surfaces, each with a faithfulness score (RQ3)	CPU	Planned
13	Choose the configuration for scale-up (RQ5)	—	Planned
14	Audit, independent re-run of a sample of results, write-up	—	Planned; drafting runs alongside

Open decisions (needed before the phase that uses them):

- The primary metric for "best" (Phase 13), declared before the main runs: e.g. mean accuracy over all corruptions, worst-family accuracy, or accuracy per CPU millisecond.
- The fine-tuning recipe (Phase 11): clean ImageNet training images of the 40 classes (a separate dataset) or the ImageNet-C training split (allowed, must be disclosed); epochs; GPU budget.

Risks:

- Kaggle resources: the weekly ~30 GPU hours and the GPU types offered (the P100 disappeared mid-project). The main runs fit in one week; fine-tuning is not yet costed.
- The ImageNet-C copy is a third-party upload, checked for completeness but not byte-for-byte — stated as a limitation.
- FastViT was trained with the same class labels (not the same images) — a confound every FastViT comparison must state.

- Training the classifier on corrupted images departs from the ImageNet-C benchmark protocol, so results are not comparable to published ImageNet-C leaderboards — required disclosure.

Files to show

All paths are inside the project folder `D:\ResearchPaper2`. For a meeting, this report plus the six plain-language documents marked ★ tell the whole story; the rest is evidence to open when a question comes up.

Order	File	What it shows
1 ★	`Documentation/01_big_picture_v1.md`	The whole project on one page: origin, aim, why
2 ★	`Documentation/03_original_paper_and_code_v1.md`	The source paper, its code and the 10 audit findings
3 ★	`Documentation/06_study_design_v1.md`	Every design decision of the study and its reason
4	`Documentation/07_choosing_the_classes_v1.md`	How the 20 similar and 20 unrelated classes were chosen
5	`Documentation/08_choosing_the_vits_v1.md`	The ViT research and why these 7 models
6	`Documentation/09_challenges_and_our_answers_v1.md`	Published criticisms of this kind of study and what was changed because of them
7	`Documentation/14_phase6_results_v3.md`	Data checks: passed twice, identical
8 ★	`Documentation/16_phase7_results_v1.md`	The 7 frozen models: speed, sanity accuracy and why
9 ★	`Documentation/17_phase8_classifier_solver_v2.md`	The classifier speed-up: every attempt, the recipe that passed, the GPU cost
10 ★	`Documentation/12_rules_and_decisions_v5.md`	Working rules and the decisions with dates

Order	File	What it shows
—	`Documentation/00_index_v10.md`	Index of all plain-language documents (latest versions)

Technical records (single files, kept current; history in git):

File	What it holds
`docs/results.md`	Every result so far with numbers: audit, port, data, features, Phase 8
`docs/plan.md`	All phases 0–14 with status and verification gates (use this, not the dated `13_whats_next_v1.md`)
`docs/scope.md`	Decision log: date, decision, who approved
`docs/project_description.md`	Research questions, data, method, required disclosures
`docs/research_vits.md`, `docs/research_counterevidence.md`	The two literature reviews behind the model choice and the added checks

Evidence to open on request:

File	What it shows
`results/phase7/phase7_summary_v3/phase7_v3/figures/`	Four charts: accuracy per model, accuracy vs compute, latency, summary vs patch vector
`results/phase8/phase8_check_v5/`	The pass that validated the exact method on the GPU (`summary.json`)
`results/phase8/phase8_check_v7/`	Re-check, speed and the GPU budget (`budget.csv`)
`results/test_reports/`	Automated test reports (latest: 155 passed)
`2501.13422v1.pdf`	The source paper

File	What it shows
`python_port/`, `study/`, `kaggle/notebooks/`	The code: corrected port, study package, Kaggle notebooks